Aula 07 — Repetição com for e range

Contagens, tabuada, percursos e repetição com quantidade definida

Dieimes Nunes

Retomando a Aula 06

Na aula anterior, iniciamos as estruturas de repetição com `while`.

Trabalhamos com:

- repetição enquanto uma condição for verdadeira;
- contadores;
- acumuladores;
- laço infinito;
- repetição até o usuário acertar;
- menus com repetição.

Exemplo:

```
contador = 1

while contador <= 5:
    print(contador)
    contador = contador + 1
```

Hoje vamos estudar outra estrutura de repetição: o `for`.

O problema da aula

Em muitos casos, sabemos exatamente quantas vezes queremos repetir algo.

Exemplos:

- mostrar os números de 1 até 10;
- repetir uma mensagem 5 vezes;
- gerar a tabuada de um número;
- ler 4 notas;
- percorrer uma sequência de valores;
- contar de 10 até 1.

Nessas situações, o `for` costuma deixar o código mais simples e organizado.

while ou for?

De forma didática, podemos pensar assim:

Use `while`

Quando a repetição depende de uma condição que pode mudar durante a execução.

Exemplo:

```
Enquanto a senha estiver errada, pedir novamente.
```

Use `for`

Quando sabemos a quantidade de repetições ou queremos percorrer uma sequência.

Exemplo:

Atividade 1 — Escolhendo while ou for

No caderno, indique se a situação combina mais com `while` ou com `for`.

1. Pedir senha até o usuário acertar.
2. Mostrar os números de 1 até 100.
3. Ler 4 notas de um aluno.
4. Manter um menu aberto até escolher sair.
5. Gerar a tabuada de um número.
6. Repetir uma mensagem 3 vezes.

Depois, explique o motivo de duas respostas.

A estrutura for

Em Python, o `for` percorre uma sequência de valores.

Estrutura básica:

```
for variavel in sequencia:
    comando_1
    comando_2
```

A variável recebe um valor da sequência a cada repetição.

Exemplo:

```
for numero in range(1, 6):
    print(numero)
```

Resultado:

```
1
2
3
```

Entendendo o range()

A função `range()` cria uma sequência de números.

Exemplo:

```
range(1, 6)
```

Gera os valores:

```
1, 2, 3, 4, 5
```

Atenção:

O último valor não entra.

Por isso, `range(1, 6)` vai até 5.

Essa é uma das primeiras pegadinhas do `for` em Python.

Primeiro exemplo com for

```
for contador in range(1, 6):
    print("Executando repetição", contador)
```

Resultado esperado:

```
Executando repetição 1
Executando repetição 2
Executando repetição 3
Executando repetição 4
Executando repetição 5
```

Nesse caso, não precisamos escrever:

```
contador = contador + 1
```

O próprio `for` avança para o próximo valor da sequência.

Atividade 2 — Primeiros testes com for

Crie três programas.

Programa 1

Mostrar os números de 1 até 10.

Programa 2

Mostrar a mensagem `Estudando Python` cinco vezes.

Programa 3

Mostrar os números de 5 até 15.

Depois, responda no caderno:

1. Qual valor inicial foi usado em cada `range()`?
2. Qual valor final foi colocado?
3. O último valor apareceu na tela?

Formas de usar range()

O `range()` pode ser usado de três formas principais.

Com um argumento

```
for i in range(5):
    print(i)
```

Resultado:

```
0
1
2
3
4
```

Quando usamos apenas um número, o Python começa em zero.

range com início e fim

```
for i in range(1, 6):
    print(i)
```

Resultado:

```
1
2
3
4
5
```

Aqui estamos informando:

```
comece em 1
pare antes de 6
```

Por isso, o último número mostrado é 5.

range com passo

Podemos informar também o passo da contagem.

```
for i in range(2, 11, 2):
    print(i)
```

Resultado:

```
2
4
6
8
10
```

A estrutura é:

```
range(início, fim, passo)
```

O passo indica de quanto em quanto a contagem deve avançar.

Atividade 3 — Entendendo o range

Sem executar, diga quais valores serão mostrados.

```
for i in range(3):
    print(i)
```

```
for i in range(2, 8):
    print(i)
```

```
for i in range(0, 11, 2):
    print(i)
```

```
for i in range(10, 0, -1):
    print(i)
```

Depois, execute no Python e confira.

Contagem decrescente com for

Para contar de forma decrescente, usamos passo negativo.

Exemplo:

```
for numero in range(10, 0, -1):
    print(numero)
```

Resultado:

```
10
9
8
7
6
5
4
3
2
1
```

Observe:

```
começa em 10
para antes de 0
anda de -1 em -1
```

Atividade 4 — Contagem regressiva

Crie um programa que mostre uma contagem regressiva de 10 até 1.

Ao final, mostre a mensagem:

```
Fim da contagem!
```

Desafio adicional:

- faça a contagem de 30 até 0, pulando de 5 em 5;
- faça a contagem de 100 até 0, pulando de 10 em 10.

Tabuada com for

A tabuada é um ótimo exemplo de repetição com quantidade definida.

```
numero = int(input("Digite um número: "))

for contador in range(1, 11):
    resultado = numero * contador
    print(f"{numero} x {contador} = {resultado}")
```

Se o usuário digitar 7, o programa mostra a tabuada do 7.

Entendendo a tabuada

Neste código:

```
numero = int(input("Digite um número: "))

for contador in range(1, 11):
    resultado = numero * contador
    print(f"{numero} x {contador} = {resultado}")
```

A variável `contador` recebe os valores de 1 até 10.

A cada repetição, o programa calcula:

```
número digitado x contador
```

Depois mostra o resultado formatado.

Atividade 5 — Tabuada

Crie um programa que peça um número ao usuário e mostre a tabuada desse número de 1 até 10.

Depois, altere o programa para mostrar:

1. tabuada de 1 até 20;
2. apenas os resultados pares da tabuada;
3. tabuada em ordem decrescente, de 10 até 1.

Atenção:

Teste com pelo menos três números diferentes.

Soma com for

Também podemos usar `for` com acumulador.

Exemplo: somar os números de 1 até 5.

```
soma = 0

for numero in range(1, 6):
    soma = soma + numero

print(f"Soma: {soma}")
```

Resultado:

```
Soma: 15
```

O funcionamento é parecido com o `while`, mas com menos controle manual do contador.

Atividade 6 — Somando uma sequência

Crie um programa que some os números de 1 até 100.

Depois, altere o programa para somar:

1. os números de 1 até 50;
2. os números pares de 2 até 100;
3. os números ímpares de 1 até 99.

No caderno, escreva:

Lendo várias notas com for

Quando sabemos a quantidade de notas, o `for` pode simplificar o programa.

```
soma = 0

for contador in range(1, 5):
    nota = float(input(f"Digite a {contador}ª nota: "))
    soma = soma + nota

media = soma / 4

print(f"Média: {media:.2f}")
```

O `range(1, 5)` executa quatro repetições:

```
1, 2, 3, 4
```

Atividade 7 — Média da turma

Crie um programa que leia a nota de 5 alunos e mostre:

- a soma das notas;
- a média da turma;
- quantas notas foram maiores ou iguais a 7.

Dicas:

- use um acumulador para somar as notas;
- use um contador para contar notas maiores ou iguais a 7;
- use `if` dentro do `for`.

Exemplo com contador dentro do for

```
aprovados = 0

for aluno in range(1, 6):
    nota = float(input(f"Digite a nota do aluno {aluno}: "))

    if nota >= 7:
        aprovados = aprovados + 1

print(f"Quantidade de aprovados: {aprovados}")
```

Esse exemplo une:

Percorrendo textos com for

O `for` também pode percorrer caracteres de um texto.

Exemplo:

```
nome = "SENAC"

for letra in nome:
    print(letra)
```

Resultado:

```
S
E
N
A
C
```

Isso mostra que o `for` não serve apenas para números.

Ele percorre sequências.

Atividade 8 — Percorrendo uma palavra

Crie um programa que peça uma palavra ao usuário e mostre uma letra por linha.

Exemplo:

```
Digite uma palavra: Python
P
y
t
h
o
n
```

Desafio adicional:

for com condição interna

Podemos combinar `for` com `if`.

Exemplo: mostrar apenas números pares.

```
for numero in range(1, 21):
    if numero % 2 == 0:
        print(numero)
```

A repetição passa por todos os números de 1 até 20.

A condição decide quais serão mostrados.

Atividade 9 — Pares e ímpares

Crie um programa que percorra os números de 1 até 30 e mostre:

- números pares;
- números ímpares;
- quantidade de pares;
- quantidade de ímpares.

Dica:

Use % para verificar o resto da divisão por 2.

```
numero % 2 == 0
```

Comparando while e for

Com while

```
contador = 1

while contador <= 5:
    print(contador)
    contador = contador + 1
```

Com for

```
for contador in range(1, 6):
    print(contador)
```

Os dois fazem a mesma contagem.

Mas o `for` é mais direto quando já sabemos a sequência.

Quando usar cada um?

Use `while` quando:

- não sabe exatamente quantas repetições acontecerão;
- depende de uma condição mudar;
- precisa manter um menu aberto;
- precisa repetir até uma resposta correta.

Use `for` quando:

- sabe a quantidade de repetições;
- precisa percorrer uma sequência;
- precisa gerar contagens;
- precisa calcular tabuada;
- precisa ler uma quantidade fixa de dados.

Erros comuns com for

Erros frequentes:

- esquecer que o último valor do `range()` não entra;
- usar passo positivo em contagem decrescente;
- confundir `range(1, 10)` com 10 repetições;
- criar acumulador dentro do laço sem necessidade;
- esquecer a indentação;
- usar o nome da variável de controle de forma confusa.

Sempre teste com valores pequenos antes de aumentar a complexidade.

Onde estamos no curso

Até aqui, já temos uma base forte de lógica:

- sequência de comandos;
- variáveis;
- entrada e saída;
- cálculos;
- comparações;
- decisões;
- condições encadeadas;
- repetição com `while`;
- repetição com `for`.

Agora já conseguimos criar programas com comportamento mais completo.

Desafio final da aula

Crie um programa de pesquisa simples.

O programa deve ler a idade de 6 pessoas e mostrar:

- a soma das idades;
- a média das idades;
- quantas pessoas são maiores de idade;
- quantas pessoas são menores de idade.

Regras:

- use `for`;
- use acumulador;
- use contadores;
- use `if` e `else`.

Fechamento

Hoje aprendemos a usar o `for` e o `range()`.

Também trabalhamos com:

- contagem crescente;
- contagem decrescente;
- passo da repetição;
- tabuada;
- soma com acumulador;
- contador dentro do laço;
- comparação entre `while` e `for`.

Na próxima aula, vamos utilizar repetições para resolver problemas um pouco mais completos, com validação, menus e estatísticas simples.